

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФБГОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики
Кафедра программного обеспечения и администрирования информационных
систем

**Проектирование базы данных
информационной системы салона красоты**

Реферат

Направление 02.03.03 «Математическое обеспечение
и администрирование информационных систем»

Зав. кафедрой _____

к. ф.-м. н., Барановский Е. С.

Обучающийся _____

2 к. 9 гр. Зяблова А. А.

Руководитель _____

ст. преп. Матвеева М. В.

Воронеж 2026

Аннотация

В реферате рассматривается проектирование базы данных для автоматизации сети салонов красоты. Построены логическая и физическая модели данных, реализованы иерархические и сетевые структуры, взаимоисключающие атрибуты, учёт исторических данных. Приведён SQL-скрипт для создания базы данных в PostgreSQL.

Содержание

Введение	4
1. Описание предметной области	5
2. Бизнес-правила	7
3. Логическая модель данных	9
3.1. Описание типов сущностей	9
3.2. Описание типов связей	12
4. Физическая модель данных	14
5. Скрипт создания базы данных	19
Заключение	20
Список литературы	21
Приложение 1. Логическая модель	22
Приложение 2. Физическая модель	24
Приложение 3. Скрипт создания базы данных в PostgreSQL	26

Введение

Данная работа посвящена проектированию базы данных для информационной системы салона красоты.

Салон красоты предоставляет различные услуги (парикмахерские, маникюр, косметологию), в процессе работы накапливаются данные о клиентах, мастерах, записях, материалах и финансах. Автоматизация этой деятельности требует создания реляционной базы данных, которая обеспечит целостность, непротиворечивость и удобный доступ к информации.

Цель работы – спроектировать логическую и физическую модели базы данных для салона красоты и реализовать SQL-скрипт для её создания. Для достижения цели выполнены следующие задачи: описание предметной области, выделение сущностей и связей, атрибутов и ограничений, построение ER-диаграммы.

1. Описание предметной области

Предметная область – сеть салонов красоты. Каждый салон находится в определённом городе, имеет адрес (улицу, дом), описание и график работы на каждый день недели. В салоне имеются помещения (комнаты/кабинеты), каждое из которых относится к определённому типу (например, «маникюрная», «парикмахерская», «косметологическая») и характеризуется номером и этажом.

В салоне работают мастера. Каждый мастер занимает определённую должность. Для каждого мастера фиксируются фамилия, имя, отчество, телефон и электронная почта. Мастер может оказывать несколько видов услуг, при этом для каждой такой специализации устанавливается индивидуальная стоимость.

Услуги классифицируются по типам (например, «Маникюр», «Стрижка», «Уход за лицом»). Каждая услуга имеет название и описание. Для оказания некоторых услуг расходуются материалы (лаки, средства для ухода, одноразовые инструменты и т.п.), их расход фиксируется.

Материалы поступают от поставщиков. Каждый материал хранится в определённых единицах измерения (штуки, миллилитры, граммы) и имеет текущий остаток на складе. Для поставки указываются дата, количество и цена товара.

Клиенты салона – физические лица. О них хранятся фамилия, имя, отчество, телефон, электронная почта и дата рождения. Клиент может оформить скидочную карту определённого типа. Каждый тип скидочной карты связан с типом услуги и содержит название, условия получения и процент скидки. У одного клиента может быть несколько карт, выданных в разные даты.

Запись на услугу содержит информацию о клиенте (и о его скидочной карте, если он ею обладает), выбранной специализации мастера, кабинете, статусе

записи, типе оплаты, дате и времени. Общая стоимость записи рассчитывается как цена услуги конкретного мастера минус скидка (если привязана карта).

2. Бизнес-правила

На основе анализа предметной области были сформулированы следующие бизнес-правила, которые определяют ограничения целостности в проектируемой базе данных.

1. Уникальность адреса салона. Адрес салона (улица, дом) в сочетании с городом должен быть уникальным. Это предотвращает дублирование информации об одном и том же салоне.

2. График работы салона. Для каждого салона можно задать время открытия и закрытия на каждый день недели. Если салон не работает в определённый день, то время открытия и закрытия не указываются. Если салон работает, то время открытия должно быть строго меньше времени закрытия.

3. График работы мастера. Каждый мастер может работать в разные дни. Для каждого рабочего дня фиксируются дата смены, время начала и окончания работы. У одного мастера не может быть двух смен в один день. Время начала смены должно быть меньше времени окончания. Дата смены не может быть в будущем (разрешены только прошедшие и текущие даты) и не может быть раньше 1900 года.

4. Специализация мастера. Мастер может оказывать несколько видов услуг, но одна и та же услуга не может быть закреплена за одним мастером дважды. Стоимость одной и той же услуги может различаться у разных мастеров, поэтому цена хранится именно на уровне специализации, а не в таблице услуг.

5. Расход материалов на услугу. Одна услуга может требовать расходования нескольких материалов. Один и тот же материал может использоваться в разных услугах. Расходуемое количество на одну услугу должно быть строго положительным.

6. *Учёт остатков материалов.* Текущий объём материала на складе не может быть отрицательным. Поступление материалов фиксируется в таблице поставок, а расход — косвенно через выполнение услуг.

7. *Занятость мастера.* В один момент времени мастер может выполнять только одну услугу.

8. *Занятость комнаты.* В одно и то же время в одном кабинете может проводиться только одна услуга.

9. *Дисконтная карта.* Каждая дисконтная карта относится к определённому типу. Тип карты привязан к типу услуги (например, карта «Маникюрная» даёт скидку только на услуги типа «Маникюр»). Процент скидки должен быть положительным числом. Номер дисконтной карты уникален. Дата выдачи карты не может быть в будущем и не может быть раньше 1900 года.

10. *Клиент.* Телефон и электронная почта клиента должны быть уникальными среди всех клиентов. Дата рождения клиента не может быть в будущем и не может быть раньше 1900 года.

11. *Поставки материалов.* В одной поставке указывается один материал от одного поставщика. Количество поставляемого материала не может быть отрицательным, цена за единицу также не может быть отрицательной.

3. Логическая модель данных

3.1. Описание типов сущностей

Логическая модель представлена в виде диаграммы «сущность-связь» (ER-диаграммы) уровня атрибутов. На диаграмме изображены все типы сущностей, их атрибуты с указанием доменов, ключей и обязательности, а также связи с указанием имён (глагольных фраз). Связи «многие-ко-многим» раскрыты через ассоциативные сущности.

Спецификация типов сущностей представлена в таблице 3.1.

Таблица 3.1.1. Типы сущностей логической модели данных

Название типа сущности	Название атрибута	Домен	Обязательный	Первичный ключ	Внешний ключ
Города/2	ID_города	Number	+	+	-
	НазваниеГорода	String	+	-	-
Салоны/1	ID_салона	Number	+	+	-
	ID_города	Number	+	-	+
	НазваниеСалона	String	+	-	-
	Улица	String	+	-	-
	Дом	String	+	-	-
	ОписаниеСалона	String	-	-	-
ТипыКабинетов/4	ID_типаКабинета	Number	+	+	-
	НазваниеТипаКабинета	String	+	-	-
	ОписаниеТипаКабинета	String	-	-	-
Кабинеты/3	ID_кабинета	Number	+	+	-
	ID_типаКабинета	Number	+	-	+
	ID_салона	Number	+	-	+
	Этаж	Number	-	-	-
	НомерКабинета	Number	+	-	-
ГрафикРаботыС/19	ID_графикаРаботыС	Number	+	+	-
	ID_салона	Number	+	-	+
	ДеньНедели	String	+	-	-
	ВремяОткрытия	Datetime	-	-	-
	ВремяЗакрытия	Datetime	-	-	-
ЕдИзмМатериалов/7	ID_едИзмерМатериала	Number	+	+	-

	ЕдиницаИзмерени я	String	+	-	-
Материалы/6	ID_материала	Number	+	+	-
	ID_едИзмерМатер иала	Number	+	-	+
	НазваниеМатериал а	String	+	-	-
	ТекКолМатериала	Number	+	-	-
Поставки/18	ID_поставки	Number	+	+	-
	ID_материала	Number	+	-	+
	ID_поставщика	Number	+	-	+
	ДатаПоставки	Datetime	+	-	-
	КоличествоМатери ала	Number	+	-	-
	СтоимостьМатериа ла	Number	+	-	-
Поставщики/5	ID_поставщика	Number	+	+	-
	НазваниеПоставщ ика	String	+	-	-
	Телефон	String	+	-	-
РасходМатериалов/2 0	ID_материала	Number	+	+	+
	ID_услуги	Number	+	+	+
	КоличествоРасхода	Number	+	-	-
Услуги/14	ID_услуги	Number	+	+	-
	ID_типаУслуги	Number	+	-	+
	НазваниеУслуги	String	+	-	-
	ОписаниеУслуги	String	-	-	-
ТипыУслуг/15	ID_типаУслуги	Number	+	+	-
	НазваниеТипаУслу ги	String	+	-	-
	ОписаниеТипаУсл уги	String	-	-	-
СпециализацииМаст ера/21	ID_специализации Ма	Number	+	+	-
	ID_услуги	Number	+	-	+
	ID_мастера	Number	+	-	+
	СтоимостьУслуги	Number	+	-	-
Мастера/8	ID_мастера	Number	+	+	-
	ID_должности	Number	+	-	+
	Фамилия	String	+	-	-
	Имя	String	+	-	-
	Отчество	String	-	-	-
	Телефон	String	+	-	-
	ЭлПочта	String	+	-	-
Должности/10	ID_должности	Number	+	+	-
	НазваниеДолжност и	String	+	-	-
	ОписаниеДолжнос ти	String	-	-	-

ГрафикРаботыМ/11	ID_графикаРаботыМ	Number	+	+	-
	ID_мастера	Number	+	-	+
	ДатаСмены	Datetime	+	-	-
	ВремяНачала	Datetime	+	-	-
	ВремяКонца	Datetime	+	-	-
Клиенты/9	ID_клиента	Number	+	+	-
	Фамилия	String	+	-	-
	Имя	String	+	-	-
	Отчество	String	-	-	-
	Телефон	String	+	-	-
	ЭлПочта	String	+	-	-
	ДатаРождения	Datetime	+	-	-
ТипыСкидКарт/12	ID_типаСкидКарты	Number	+	+	-
	ID_типаУслуги	Number	+	-	+
	НазваниеТипаСкидК	String	+	-	-
	УсловияПолучения	String	+	-	-
	ПроцентСкидки	Number	+	-	-
СкидочныеКарты/13	ID_скидКарты	Number	+	+	-
	ID_типаСкидКарты	Number	+	-	+
	ID_клиента	Number	+	-	+
	НомерСкидКарты	String	+	-	-
	ДатаВыдачи	Datetime	+	-	-
Записи/16	ID_записи	Number	+	+	-
	ID_типаОплаты	Number	+	-	+
	ID_скидКарты	Number	-	-	+
	ID_кабинета	Number	+	-	+
	ID_специализацииМ	Number	+	-	+
	ID_клиента	Number	-	-	+
	ID_статусаЗаписи	Number	+	-	+
	ДатаЗаписи	Datetime	+	-	-
	ВремяЗаписи	Datetime	+	-	-
	Стоимость	Number	+	-	-
СтатусыЗаписей/17	ID_статусаЗаписи	Number	+	+	-
	НазваниеСтатуса	String	+	-	-
ТипыОплаты/22	ID_типаОплаты	Number	+	+	-
	НазваниеТипаОплаты	String	+	-	-

В типе сущности «Записи» атрибуты ID_клиента и ID_скидКарты являются взаимоисключающими: запись может ссылаться либо на клиента, либо на дисконтную карту, но не на оба атрибута одновременно.

3.2. Описание типов связей

Типы связей между сущностями логической модели описаны в таблице 3.2.1.

Таблица 3.2.1. Типы связей между сущностями логической модели

Связываемые сущности	Тип связи	Идентифицирующая	Название связи
Города – Салоны	Один-ко-многим	-	содержат/находятся в
Салоны – Кабинеты	Один-ко-многим	-	содержат/находятся в
Салоны – ГрафикРаботыС	Один-ко-многим	-	имеют/относятся
ТипыКабинетов – Кабинеты	Один-ко-многим	-	характеризуют/соответствуют
Должности – Мастера	Один-ко-многим	-	классифицируют/обладают
Мастера – ГрафикРаботыМ	Один-ко-многим	-	работают по/характеризует
Мастера – СпециализацияМастера	Один-ко-многим	-	обладают/характеризуют
Услуги – СпециализацияМастера	Один-ко-многим	-	определяют/включают
ТипыУслуг – Услуги	Один-ко-многим	-	классифицируют/соответствуют
ТипыУслуг – ТипыСкидКарт	Один-ко-многим	-	классифицируют/соответствуют
Клиенты – СкидочныеКарты	Один-ко-многим	-	обладают/относятся к
ТипыСкидКарт – СкидочныеКарты	Один-ко-многим	-	характеризуют/относятся к
ЕдИзмМатериалов – Материалы	Один-ко-многим	-	характеризуют/используют
Материалы – Поставки	Один-ко-многим	-	пополняются через/поставляют
Поставщики – Поставки	Один-ко-многим	-	осуществляют/содержат
Материалы – РасходМатериалов	Один-ко-многим	+	включаются в/используют
Услуги – РасходМатериалов	Один-ко-многим	+	определяют/соответствуют
Кабинеты – Записи	Один-ко-многим	-	характеризуют/содержат
СпециализацииМастера – Записи	Один-ко-многим	-	обладают/включают
Клиенты – Записи	Один-ко-многим	-	оформляют/принадлежат
СкидочныеКарты – Записи	Один-ко-многим	-	используются в/учитывают
СтатусыЗаписей – Записи	Один-ко-многим	-	характеризуют/соответствуют
ТипыОплаты – Записи	Один-ко-многим	-	описывают/оплачиваются

В модели присутствуют три связи «многие-ко-многим», которые раскрыты через ассоциативные сущности:

1. Мастера – Услуги. Раскрыта через сущность «СпециализацииМастера», которая также хранит стоимость услуги для конкретного мастера.
2. Материалы – Услуги. Раскрыта через сущность «РасходМатериалов», которая хранит количество материала, необходимое для выполнения услуги.
3. Поставщики – Материалы. Раскрыта через сущность «Поставки», которая фиксирует дату, количество и цену каждой поставки.

4. Физическая модель данных

Физическая модель описывает реализацию базы данных в конкретной СУБД (PostgreSQL). Ниже в таблице 4.1 представлено описание каждой таблицы: её назначение, столбцы, типы данных и ограничения.

Таблица 4.1. Описание таблиц базы данных (физическая модель)

Название таблицы	Название столбца	Тип	Ограничения
cities_2 (хранение перечня городов, в которых расположены салоны)	id_city	SERIAL	PRIMARY KEY
	name_city	TEXT	NOT NULL, UNIQUE
salons_1 (хранение информации о салонах красоты)	id_salon	SERIAL	PRIMARY KEY
	id_city	INTEGER	NOT NULL, FOREIGN KEY (cities)
	name_salon	TEXT	NOT NULL
	street	TEXT	NOT NULL
	building	TEXT	NOT NULL
	description_salon	TEXT	-
room_types_4 (хранение типов помещений)	id_room_type	SERIAL	PRIMARY KEY
	name_room_type	TEXT	NOT NULL, UNIQUE
	description_room_type	TEXT	-
rooms_3 (хранение сведений о помещениях (кабинетах) в салонах)	id_room	SERIAL	PRIMARY KEY
	id_room_type	INTEGER	NOT NULL, FOREIGN KEY (salons)
	id_salon	INTEGER	NOT NULL, FOREIGN KEY (room_types)
	room_floor	INTEGER	-
	room_number	TEXT	NOT NULL
salon_schedules_19 (хранение графика работы каждого салона по дням недели)	id_schedule_salon	SERIAL	PRIMARY KEY
	id_salon	INTEGER	NOT NULL, FOREIGN KEY (salons)
	day_of_week	CHAR(1)	NOT NULL, CHECK (IN ('1','2','3','4','5','6','7'))
	open_time	TIME	CHECK((open_time IS NULL AND close_time IS NULL) OR (open_time IS NOT NULL AND close_time IS NOT NULL AND open_time < close_time))
	close_time	TIME	
measurement_units_7 (хранение единиц измерения для материалов)	id_measurement_unit	SERIAL	PRIMARY KEY
	name_measurement_unit	TEXT	NOT NULL, UNIQUE
materials_6	id_material	SERIAL	PRIMARY KEY

(хранение информации о расходных материалах)	id_measurement_unit	INTEGER	NOT NULL, FOREIGN KEY (measurement_units)
	name_material	TEXT	NOT NULL, UNIQUE
	current_volume_material	INTEGER	NOT NULL, CHECK (current_volume_material >= 0)
supplies_18 (хранение записей о поступлении материалов от поставщиков)	id_supply	SERIAL	PRIMARY KEY
	id_material	INTEGER	NOT NULL, FOREIGN KEY (materials)
	id_supplier	INTEGER	NOT NULL, FOREIGN KEY (suppliers)
	date_supply	DATE	NOT NULL
	quantity_material	INTEGER	NOT NULL, CHECK (quantity_material > 0)
	price_material	MONEY	NOT NULL, CHECK (price_material >= 0)
suppliers_5 (хранение информации о поставщиках материалов)	id_supplier	SERIAL	PRIMARY KEY
	name_supplier	TEXT	NOT NULL, UNIQUE
	phone_supplier	TEXT	NOT NULL, UNIQUE
material_consumptions_20 (связь материалов и услуг (раскрытие связи многие-ко-многим), хранение нормы расхода материала на услугу)	id_material	INTEGER	NOT NULL, FOREIGN KEY (materials)
	id_service	INTEGER	NOT NULL, FOREIGN KEY (services)
	consumed_quantity	INTEGER	NOT NULL, CHECK (consumed_quantity > 0)
services_14 (хранение перечня услуг, предоставляемых в салоне)	id_service	SERIAL	PRIMARY KEY
	id_service_type	INTEGER	NOT NULL, FOREIGN KEY (service_types)
	name_service	TEXT	NOT NULL, UNIQUE
	description_service	TEXT	-
services_types_15 (хранение категорий услуг)	id_service_type	SERIAL	PRIMARY KEY
	name_service_type	TEXT	NOT NULL, UNIQUE
	description_service_type	TEXT	-
master_specializations_21 (хранение стоимости услуги у конкретного мастера)	id_master_specialization	SERIAL	PRIMARY KEY
	id_master	INTEGER	NOT NULL, FOREIGN KEY (masters)
	id_service	INTEGER	NOT NULL, FOREIGN KEY (services)
	price_service	MONEY	NOT NULL, CHECK (price_service >= 0)
masters_8 (хранение информации о мастерах (сотрудниках салона))	id_master	SERIAL	PRIMARY KEY
	id_position	INTEGER	NOT NULL, FOREIGN KEY (positions)
	surname_master	TEXT	NOT NULL
	name_master	TEXT	NOT NULL
	patronymic_master	TEXT	-
	phone_master	TEXT	NOT NULL, UNIQUE
	email_master	TEXT	NOT NULL, UNIQUE
positions_10 (хранение должностей сотрудников)	id_position	SERIAL	PRIMARY KEY
	name_position	TEXT	NOT NULL, UNIQUE
	description_position	TEXT	-
master_schedules_11	id_schedule_master	SERIAL	PRIMARY KEY

(хранение расписания рабочих смен мастеров (исторические данные))	id_master	INTEGER	NOT NULL, FOREIGN KEY (masters)
	date_shift	DATE	NOT NULL, CHECK (date_shift >= '1900-01-01' AND date_shift <= CURRENT_DATE)
	start_time	TIME	NOT NULL
	end_time	TIME	NOT NULL, CHECK (start_time < end_time)
clients_9 (хранение информации о клиентах салона)	id_client	SERIAL	PRIMARY KEY
	surname_client	TEXT	NOT NULL
	name_client	TEXT	NOT NULL
	patronymic_client	TEXT	-
	phone_client	TEXT	NOT NULL, UNIQUE
	email_client	TEXT	NOT NULL, UNIQUE
	birth_date_client	DATE	NOT NULL, CHECK (birth_date_client >= '1900-01-01' AND birth_date_client <= CURRENT_DATE)
discount_card_types_12 (хранение типов дисконтных карт с привязкой к типу услуги)	id_discount_card_type	SERIAL	PRIMARY KEY
	id_service_type	INTEGER	NOT NULL, FOREIGN KEY (service_types)
	name_discount_card_type	TEXT	NOT NULL, UNIQUE
	requirements	TEXT	-
	discount_percentage	INTEGER	NOT NULL, CHECK (discount_percentage > 0)
discount_cards_13 (хранение выданных клиентам дисконтных карт)	id_discount_card	SERIAL	PRIMARY KEY
	id_client	INTEGER	NOT NULL, FOREIGN KEY (clients)
	id_discount_card_type	INTEGER	NOT NULL, FOREIGN KEY (discount_card_types)
	number_discount_card	TEXT	NOT NULL, UNIQUE
	date_issue	DATE	NOT NULL, CHECK (date_issue >= '1900-01-01' AND date_issue <= CURRENT_DATE)
appointments_16 (хранение записей клиентов на услуги)	id_appointment	SERIAL	PRIMARY KEY
	id_client	INTEGER	NOT NULL, FOREIGN KEY (clients)
	id_master_specialization	INTEGER	NOT NULL, FOREIGN KEY (master_specializations)
	id_room	INTEGER	NOT NULL, FOREIGN KEY (rooms)
	id_discount_card	INTEGER	NULL, FOREIGN KEY (discount_cards)
	id_payment_type	INTEGER	NOT NULL, FOREIGN KEY (payment_types)
	id_appointment_status	INTEGER	NOT NULL, FOREIGN KEY (appointment_statuses)
	date_appointment	DATE	NOT NULL (date_appointment >= '1900-01-01' AND date_appointment <= CURRENT_DATE)
	appointment_time	TIME	NOT NULL

	total_cost	MONEY	NOT NULL, CHECK (total_cost >= 0)
appointment_statuses_17 (хранение возможных статусов записи (подтверждена, выполнена, отменена и т.п.))	id_appointment_status	SERIAL	PRIMARY KEY
	name_appointment_status	TEXT	NOT NULL, UNIQUE
payment_types_22 (хранение возможных способов оплаты (наличные, карта, перевод))	id_payment_type	SERIAL	PRIMARY KEY
	name_payment_type	TEXT	NOT NULL, UNIQUE

Некоторые ограничения невозможно задать в рамках одного столбца, так как они охватывают два и более столбца или содержат сложную проверку. Перечень таких ограничений представлен в таблице 4.2.

Таблица 4.2. Составные ограничения

Таблица	Ограничение	Тип ограничения
salons	UNIQUE (street, building, id_city)	Составная уникальность
salon_schedules	CHECK ((open_time IS NULL AND close_time IS NULL) OR (open_time IS NOT NULL AND close_time IS NOT NULL AND open_time < close_time))	Проверка, что время открытия меньше времени закрытия
rooms	UNIQUE (id_salon, room_number, room_floor)	Составная уникальность
master_schedules	UNIQUE (id_master, date_shift)	Составная уникальность
master_schedules	CHECK (start_time < end_time)	Проверка, что время начала смены меньше времени конца смены
appointments	CHECK ((id_client IS NULL AND id_discount_card IS NOT NULL) OR (id_client IS NOT NULL AND id_discount_card IS NULL))	Взаимоисключение
appointments	UNIQUE (id_master_specialization, date_appointment, appointment_time)	Составная уникальность (занятость мастера)

appointments	UNIQUE (id_room, date_appointment, appointment_time)	Составная уникальность (занятость комнаты)
master_specializations	UNIQUE (id_master, id_service)	Составная уникальность
material_consumptions	PRIMARY KEY (id_material, id_service)	Составной первичный ключ

5. Скрипт создания базы данных

На основе разработанной физической модели был составлен SQL-скрипт для создания базы данных в СУБД PostgreSQL. Скрипт решает следующие задачи:

1. последовательное создание всех таблиц в порядке, учитывающем ссылочную целостность (таблицы, на которые ссылаются внешние ключи, создаются раньше);
2. определение первичных ключей для обеспечения уникальности записей;
3. определение внешних ключей для поддержания ссылочной целостности данных;
4. задание ограничений уникальности для недопущения дублирования значений (например, номера телефона клиента, названия услуги);
5. задание проверочных ограничений (CHECK) для контроля допустимых значений (например, положительная стоимость услуги, корректный диапазон дат);
6. реализация взаимоисключающей связи между клиентом и дисконтной картой в таблице записей;
7. реализация составных уникальных ограничений для контроля занятости мастеров и комнат в одно и то же время.

Перед созданием таблиц скрипт содержит команды `DROP TABLE IF EXISTS ... CASCADE`, которые удаляют существующие таблицы (если они есть) вместе с зависимыми объектами. Это позволяет пересоздавать базу данных без возникновения ошибок.

Полный листинг скрипта приведён в Приложении.

Заключение

В ходе выполнения данной работы была спроектирована база данных для информационной системы салона красоты.

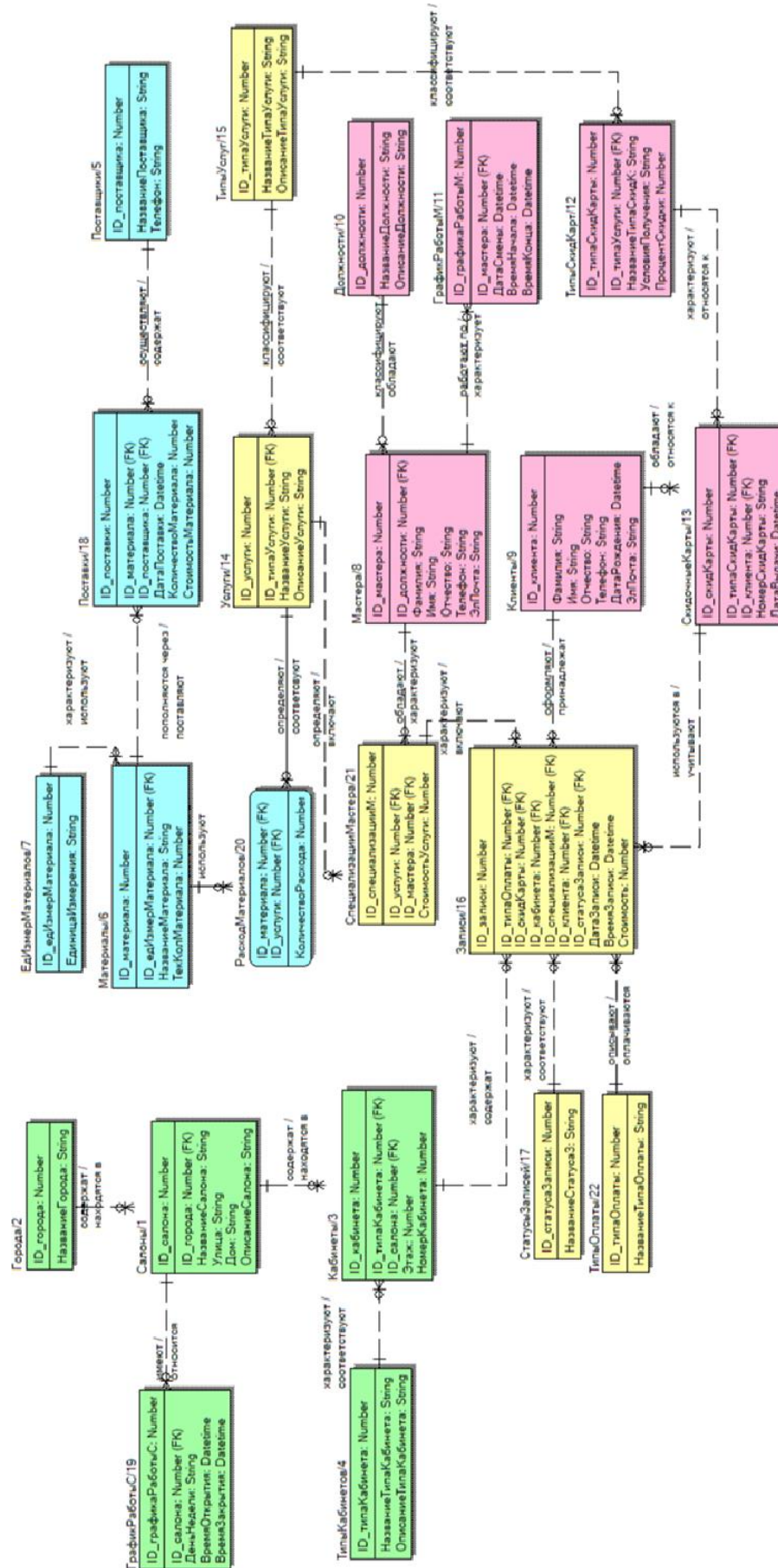
Была проанализирована предметная область, сформулированы бизнес-правила, выделены основные сущности и связи между ними. Построена логическая модель данных, включающая описание типов сущностей, их атрибутов и типов связей. На основе логической модели разработана физическая модель, содержащая структуру таблиц, типы данных и ограничения целостности. Составлен рабочий SQL-скрипт для создания базы данных в СУБД PostgreSQL.

Спроектированная база данных может быть использована в качестве основы для разработки автоматизированной информационной системы управления салоном красоты.

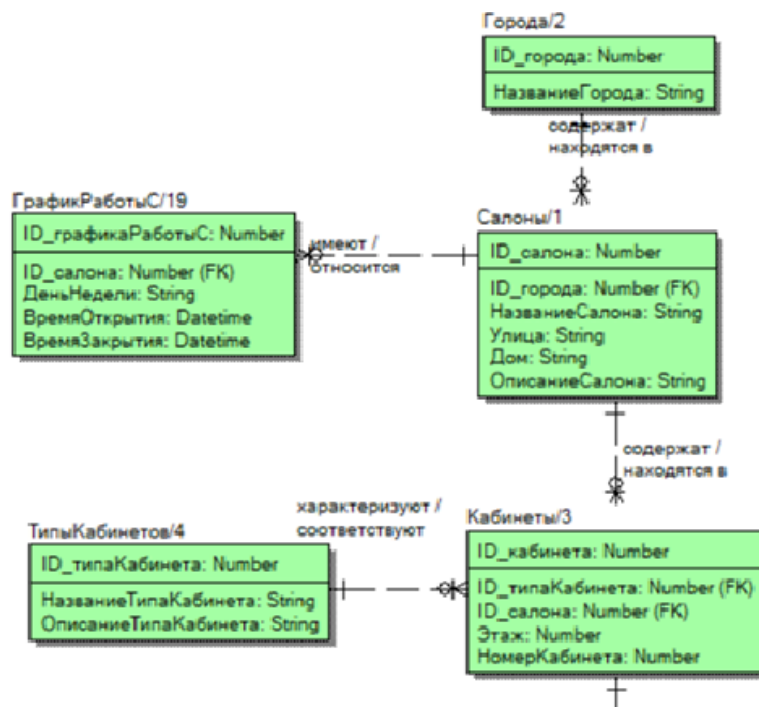
Список литературы

1. Воронина И. Е. Структура, содержание и оформление бакалаврской работы : учеб.-метод. пособие / И. Е. Воронина, И. И. Каширская, М. В. Матвеева. – Воронеж : Издательский дом ВГУ, 2025. – 88 с.
2. Левшин И. В. Postgres. Первое знакомство / И. В. Левшин, П. В. Лузанов, Е. В. Рогов. – М. : Постгрес Профессиональный, 2020. – 178 с. – ISBN 978-5-6041193-6-5.
3. Комаров В. И. Путеводитель по базам данных : учеб. пособие / В. И. Комаров. – Москва : ДМК Пресс, 2024. – 520 с. – ISBN 978-5-93700-287-7.

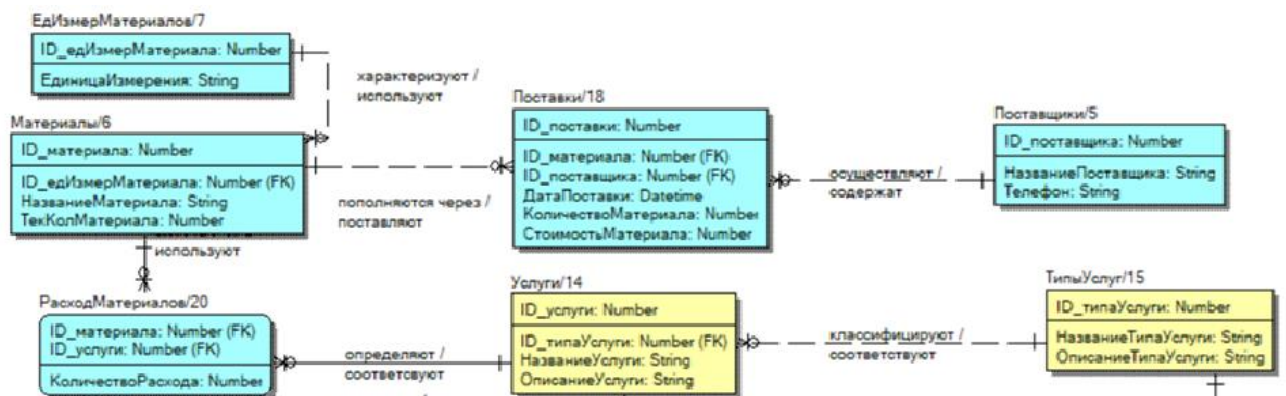
Приложение 1. Логическая модель



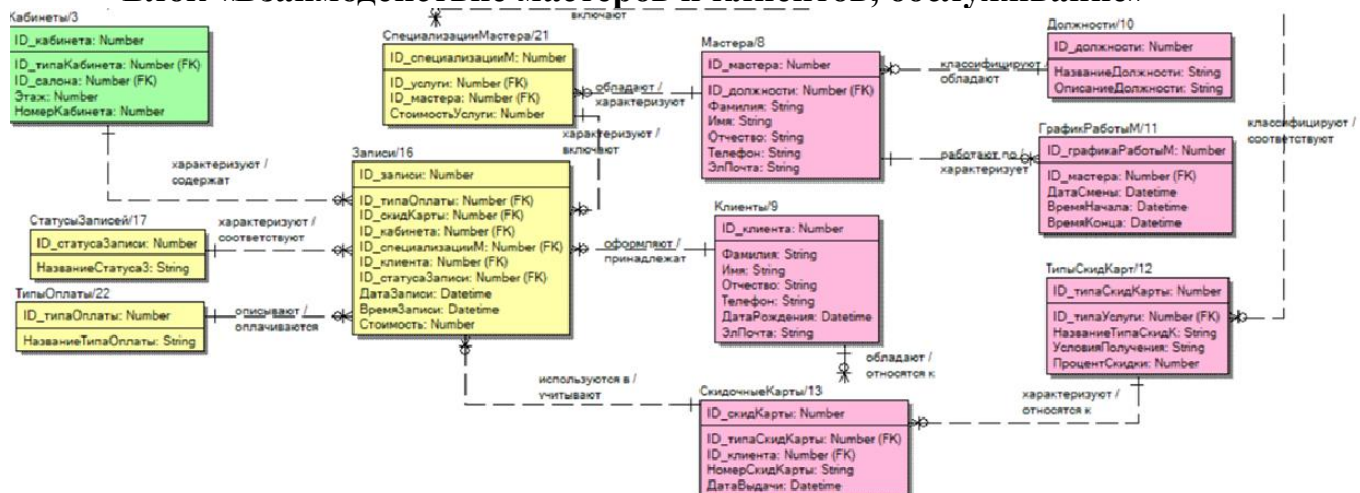
Блок «Салоны»



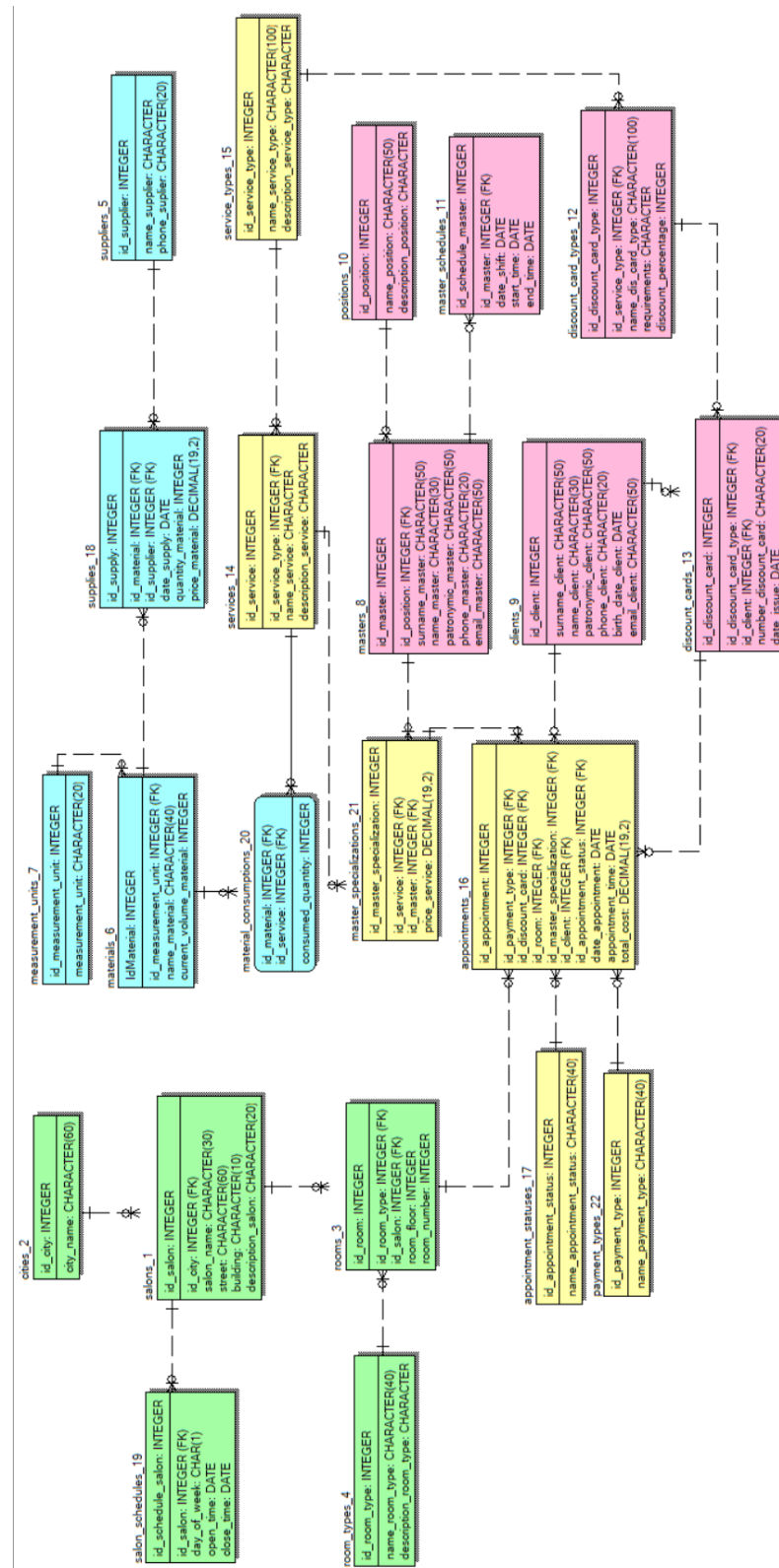
Блок «Материалы»



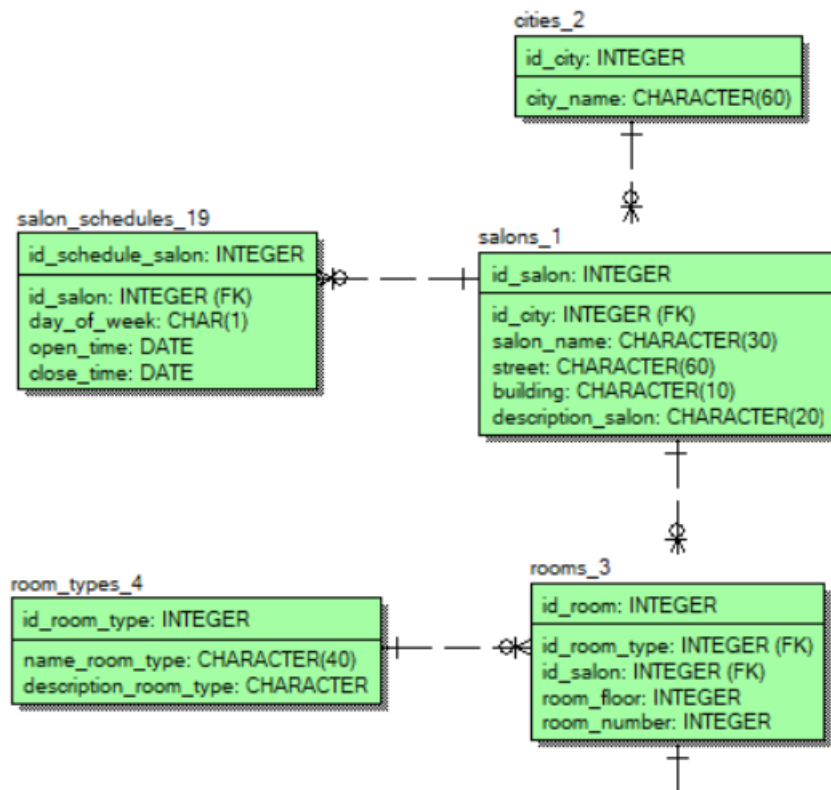
Блок «Взаимодействие мастеров и клиентов, обслуживание»



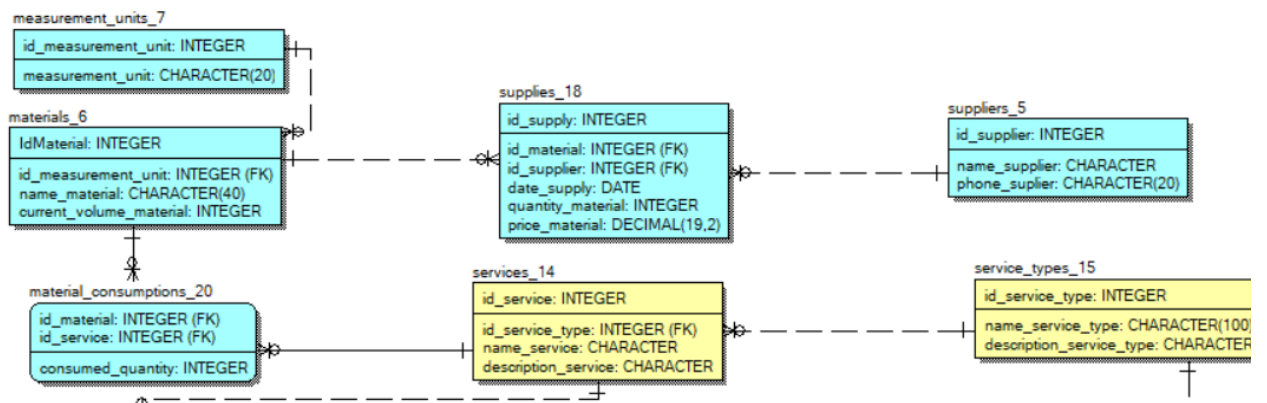
Приложение 2. Физическая модель



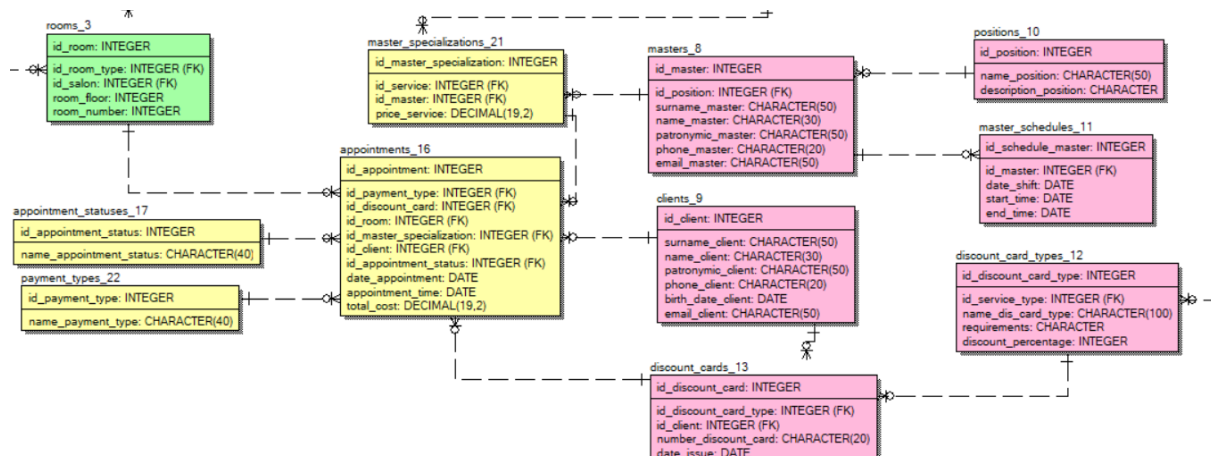
Блок «Салоны»



Блок «Материалы»



Блок «Взаимодействие мастеров и клиентов, обслуживание»



Приложение 3. Скрипт создания базы данных в PostgreSQL

```
DROP TABLE IF EXISTS material_consumptions CASCADE;
DROP TABLE IF EXISTS master_specializations CASCADE;
DROP TABLE IF EXISTS supplies CASCADE;
DROP TABLE IF EXISTS salon_schedules CASCADE;
DROP TABLE IF EXISTS appointments CASCADE;
DROP TABLE IF EXISTS discount_cards CASCADE;
DROP TABLE IF EXISTS master_schedules CASCADE;
DROP TABLE IF EXISTS rooms CASCADE;
DROP TABLE IF EXISTS salons CASCADE;
DROP TABLE IF EXISTS clients CASCADE;
DROP TABLE IF EXISTS masters CASCADE;
DROP TABLE IF EXISTS materials CASCADE;
DROP TABLE IF EXISTS suppliers CASCADE;
DROP TABLE IF EXISTS services CASCADE;
DROP TABLE IF EXISTS measurement_units CASCADE;
DROP TABLE IF EXISTS service_types CASCADE;
DROP TABLE IF EXISTS discount_card_types CASCADE;
DROP TABLE IF EXISTS appointment_statuses CASCADE;
DROP TABLE IF EXISTS payment_types CASCADE;
DROP TABLE IF EXISTS positions CASCADE;
DROP TABLE IF EXISTS room_types CASCADE;
DROP TABLE IF EXISTS cities CASCADE;
```

```
CREATE TABLE salons(
    id_salon SERIAL NOT NULL,
    salon_name TEXT NOT NULL,
    street TEXT NOT NULL,
    building TEXT NOT NULL,
    description_salon TEXT,
    id_city INTEGER NOT NULL
);

ALTER TABLE salons ADD CONSTRAINT pk_id_salon PRIMARY KEY(id_salon);
ALTER TABLE salons ADD CONSTRAINT u_street_building_id_city UNIQUE (street,
building, id_city);
```

```

CREATE TABLE salon_schedules (
    id_schedule_salon SERIAL NOT NULL,
    id_salon INTEGER NOT NULL,
    day_of_week CHAR(1) NOT NULL,
    open_time TIME,
    close_time TIME
);
ALTER TABLE salon_schedules ADD CONSTRAINT pk_id_schedule_salon
PRIMARY KEY (id_schedule_salon);
ALTER TABLE salon_schedules ADD CONSTRAINT ch_salon_schedule_hours CHECK (
    ((open_time IS NULL
        AND close_time IS NULL)
    OR (open_time IS NOT NULL
        AND close_time IS NOT NULL
        AND open_time < close_time));
ALTER TABLE salon_schedules ADD CONSTRAINT ch_salon_schedule_day CHECK (
    day_of_week IN ('1', '2', '3', '4', '5', '6', '7'));

```

```

CREATE TABLE rooms (
    id_room SERIAL NOT NULL,
    id_salon INTEGER NOT NULL,
    id_room_type INTEGER NOT NULL,
    room_number TEXT NOT NULL,
    room_floor INTEGER
);
ALTER TABLE rooms ADD CONSTRAINT pk_id_room PRIMARY KEY (id_room);
ALTER TABLE rooms ADD CONSTRAINT u_room_number_in_salon UNIQUE (id_salon,
room_number, room_floor);

```

```

CREATE TABLE masters (
    id_master SERIAL NOT NULL,
    id_position INTEGER NOT NULL,
    surname_master TEXT NOT NULL,
    name_master TEXT NOT NULL,
    patronymic_master TEXT,
    phone_master TEXT NOT NULL,

```

```

email_master TEXT NOT NULL
);
ALTER TABLE masters ADD CONSTRAINT pk_id_master PRIMARY KEY (id_master);
ALTER TABLE masters ADD CONSTRAINT u_phone_master UNIQUE (phone_master);
ALTER TABLE masters ADD CONSTRAINT u_email_master UNIQUE (email_master);

CREATE TABLE master_schedules (
    id_schedule_master SERIAL NOT NULL,
    id_master INTEGER NOT NULL,
    date_shift DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL
);
ALTER TABLE master_schedules ADD CONSTRAINT pk_id_schedule_master
PRIMARY KEY (id_schedule_master);
ALTER TABLE master_schedules ADD CONSTRAINT u_master_schedule_day
UNIQUE (id_master, date_shift);
ALTER TABLE master_schedules ADD CONSTRAINT ch_date_shift CHECK (
    date_shift >= DATE '1900-01-01'
    AND date_shift <= CURRENT_DATE);
ALTER TABLE master_schedules ADD CONSTRAINT ch_start_time_end_time
CHECK (start_time < end_time);

CREATE TABLE master_specializations (
    id_master_specialization SERIAL NOT NULL,
    id_master INTEGER NOT NULL,
    id_service INTEGER NOT NULL,
    price_service MONEY NOT NULL
);
ALTER TABLE master_specializations ADD CONSTRAINT pk_id_master_specialization
PRIMARY KEY (id_master_specialization);
ALTER TABLE master_specializations ADD CONSTRAINT u_master_service
UNIQUE (id_master, id_service);
ALTER TABLE master_specializations ADD CONSTRAINT ch_price_service
CHECK (price_service >= 0::MONEY);

```

```

CREATE TABLE clients (
    id_client SERIAL NOT NULL,
    surname_client TEXT NOT NULL,
    name_client TEXT NOT NULL,
    patronymic_client TEXT,
    phone_client TEXT NOT NULL,
    email_client TEXT NOT NULL,
    birth_date_client DATE NOT NULL
);
ALTER TABLE clients ADD CONSTRAINT pk_id_client PRIMARY KEY (id_client);
ALTER TABLE clients ADD CONSTRAINT u_phone_client UNIQUE (phone_client);
ALTER TABLE clients ADD CONSTRAINT u_email_client UNIQUE (email_client);
ALTER TABLE clients ADD CONSTRAINT ch_birth_date_client
CHECK (
    birth_date_client >= DATE '1900-01-01'
    AND birth_date_client <= CURRENT_DATE);

```

```

CREATE TABLE materials (
    id_material SERIAL NOT NULL,
    id_measurement_unit INTEGER NOT NULL,
    name_material TEXT NOT NULL,
    current_volume_material INTEGER NOT NULL
);
ALTER TABLE materials ADD CONSTRAINT pk_id_material PRIMARY KEY (id_material);
ALTER TABLE materials ADD CONSTRAINT u_name_material UNIQUE (name_material);
ALTER TABLE materials ADD CONSTRAINT ch_current_volume_material
CHECK (current_volume_material >= 0);

```

```

CREATE TABLE services (
    id_service SERIAL NOT NULL,
    id_service_type INTEGER NOT NULL,
    name_service TEXT NOT NULL,
    description_service TEXT
);
ALTER TABLE services ADD CONSTRAINT pk_id_service PRIMARY KEY (id_service);
ALTER TABLE services ADD CONSTRAINT u_name_service UNIQUE (name_service);

```

```

CREATE TABLE discount_cards (
    id_discount_card SERIAL NOT NULL,
    id_client INTEGER NOT NULL,
    id_discount_card_type INTEGER NOT NULL,
    number_discount_card TEXT NOT NULL,
    date_issue DATE NOT NULL
);

ALTER TABLE discount_cards ADD CONSTRAINT pk_id_discount_card
PRIMARY KEY (id_discount_card);

ALTER TABLE discount_cards ADD CONSTRAINT u_number_discount_card
UNIQUE (number_discount_card);

ALTER TABLE discount_cards ADD CONSTRAINT ch_date_issue
CHECK (
    date_issue >= DATE '1900-01-01'
    AND date_issue <= CURRENT_DATE);

CREATE TABLE appointments (
    id_appointment SERIAL NOT NULL,
    id_client INTEGER,
    id_master_specialization INTEGER NOT NULL,
    id_room INTEGER NOT NULL,
    id_appointment_status INTEGER NOT NULL,
    id_payment_type INTEGER NOT NULL,
    id_discount_card INTEGER,
    date_appointment DATE NOT NULL,
    appointment_time TIME NOT NULL,
    total_cost MONEY NOT NULL
);

ALTER TABLE appointments ADD CONSTRAINT pk_id_appointment
PRIMARY KEY (id_appointment);

ALTER TABLE appointments ADD CONSTRAINT u_appointment_master_time
UNIQUE (id_master_specialization, date_appointment, appointment_time);

ALTER TABLE appointments ADD CONSTRAINT u_appointment_room_time
UNIQUE (id_room, date_appointment, appointment_time);

ALTER TABLE appointments ADD CONSTRAINT ch_id_discount_card_id_client

```

```

CHECK (
    (id_client IS NULL
    AND id_discount_card IS NOT NULL)
    OR
    (id_client IS NOT NULL
    AND id_discount_card IS NULL)
);
ALTER TABLE appointments ADD CONSTRAINT ch_total_cost
CHECK (total_cost >= 0::MONEY);
ALTER TABLE appointments ADD CONSTRAINT ch_date_appointment
CHECK (date_appointment >= '1900-01-01' AND date_appointment <= CURRENT_DATE);

CREATE TABLE supplies (
    id_supply SERIAL NOT NULL,
    id_material INTEGER NOT NULL,
    id_supplier INTEGER NOT NULL,
    date_supply DATE NOT NULL,
    quantity_material INTEGER NOT NULL,
    price_material MONEY NOT NULL
);
ALTER TABLE supplies ADD CONSTRAINT pk_id_supply PRIMARY KEY (id_supply);
ALTER TABLE supplies ADD CONSTRAINT ch_price_material
CHECK (price_material >= 0::MONEY);

CREATE TABLE material_consumptions (
    id_material INTEGER NOT NULL,
    id_service INTEGER NOT NULL,
    consumed_quantity INTEGER NOT NULL
);
ALTER TABLE material_consumptions ADD CONSTRAINT pk_id_material_id_service
PRIMARY KEY (id_material, id_service);
ALTER TABLE material_consumptions ADD CONSTRAINT ch_consumed_quantity
CHECK (consumed_quantity > 0);

CREATE TABLE suppliers (
    id_supplier SERIAL NOT NULL,

```

```

    name_supplier TEXT NOT NULL,
    phone_supplier TEXT NOT NULL
);
ALTER TABLE suppliers ADD CONSTRAINT pk_id_supplier PRIMARY KEY (id_supplier);
ALTER TABLE suppliers ADD CONSTRAINT u_phone_supplier UNIQUE (phone_supplier);
ALTER TABLE suppliers ADD CONSTRAINT u_name_supplier UNIQUE (name_supplier);

CREATE TABLE cities (
    id_city SERIAL NOT NULL,
    name_city TEXT NOT NULL
);
ALTER TABLE cities ADD CONSTRAINT pk_id_city PRIMARY KEY (id_city);
ALTER TABLE cities ADD CONSTRAINT u_name_city UNIQUE (name_city);

CREATE TABLE room_types (
    id_room_type SERIAL NOT NULL,
    name_room_type TEXT NOT NULL,
    description_room_type TEXT
);
ALTER TABLE room_types ADD CONSTRAINT pk_id_room_type PRIMARY KEY
(id_room_type);
ALTER TABLE room_types ADD CONSTRAINT u_name_room_type UNIQUE
(name_room_type);

CREATE TABLE positions (
    id_position SERIAL NOT NULL,
    name_position TEXT NOT NULL,
    description_position TEXT
);
ALTER TABLE positions ADD CONSTRAINT pk_id_position PRIMARY KEY (id_position);
ALTER TABLE positions ADD CONSTRAINT u_name_position UNIQUE (name_position);

CREATE TABLE measurement_units (
    id_measurement_unit SERIAL NOT NULL,
    name_measurement_unit TEXT NOT NULL
);
ALTER TABLE measurement_units ADD CONSTRAINT pk_id_measurement_unit

```

```
PRIMARY KEY (id_measurement_unit);
ALTER TABLE measurement_units ADD CONSTRAINT u_name_measurement_unit
UNIQUE (name_measurement_unit);
```

```
CREATE TABLE service_types (
    id_service_type SERIAL NOT NULL,
    name_service_type TEXT NOT NULL,
    description_service_type TEXT
);
ALTER TABLE service_types ADD CONSTRAINT pk_id_service_type\
PRIMARY KEY (id_service_type);
ALTER TABLE service_types ADD CONSTRAINT u_name_service_type
UNIQUE (name_service_type);
```

```
CREATE TABLE discount_card_types (
    id_discount_card_type SERIAL NOT NULL,
    id_service_type INTEGER NOT NULL,
    name_discount_card_type TEXT NOT NULL,
    requirements TEXT,
    discount_percentage INTEGER NOT NULL
);
ALTER TABLE discount_card_types ADD CONSTRAINT pk_id_discount_card_type
PRIMARY KEY (id_discount_card_type);
ALTER TABLE discount_card_types ADD CONSTRAINT u_name_discount_card_type
UNIQUE (name_discount_card_type);
ALTER TABLE discount_card_types ADD CONSTRAINT ch_discount_percentage
CHECK (discount_percentage > 0);
```

```
CREATE TABLE appointment_statuses (
    id_appointment_status SERIAL NOT NULL,
    name_appointment_status TEXT NOT NULL
);
ALTER TABLE appointment_statuses ADD CONSTRAINT pk_id_appointment_status
PRIMARY KEY (id_appointment_status);
ALTER TABLE appointment_statuses ADD CONSTRAINT u_name_appointment_status
UNIQUE (name_appointment_status);
```

```

CREATE TABLE payment_types (
    id_payment_type SERIAL NOT NULL,
    name_payment_type TEXT NOT NULL
);
ALTER TABLE payment_types ADD CONSTRAINT pk_id_payment_type PRIMARY KEY
(id_payment_type);
ALTER TABLE payment_types ADD CONSTRAINT u_name_payment_type UNIQUE
(name_payment_type);
-- ===== FOREIGN KEYS =====
-- Salons
ALTER TABLE salons ADD CONSTRAINT FK_salons_id_city FOREIGN KEY (id_city)
REFERENCES cities (id_city);

-- Rooms
ALTER TABLE rooms ADD CONSTRAINT FK_rooms_id_salon FOREIGN KEY (id_salon)
REFERENCES salons (id_salon);
ALTER TABLE rooms ADD CONSTRAINT FK_rooms_id_room_type
FOREIGN KEY (id_room_type)
REFERENCES room_types (id_room_type);

-- Masters
ALTER TABLE masters ADD CONSTRAINT FK_masters_id_position
FOREIGN KEY (id_position)
REFERENCES positions (id_position);

-- MasterSchedules
ALTER TABLE master_schedules ADD CONSTRAINT FK_master_schedules_id_master
FOREIGN KEY (id_master)
REFERENCES masters (id_master);

-- MasterSpecializations
ALTER TABLE master_specializations ADD CONSTRAINT
FK_master_specializations_id_master FOREIGN KEY (id_master)
REFERENCES masters (id_master);

ALTER TABLE master_specializations ADD CONSTRAINT
FK_master_specializations_id_service FOREIGN KEY (id_service)

```

```
REFERENCES services (id_service);
```

```
-- Materials
```

```
ALTER TABLE materials ADD CONSTRAINT FK_materials_id_measurement_unit
FOREIGN KEY (id_measurement_unit)
REFERENCES measurement_units (id_measurement_unit);
```

```
-- MaterialConsumptions
```

```
ALTER TABLE material_consumptions ADD CONSTRAINT
FK_material_consumptions_id_material FOREIGN KEY (id_material)
REFERENCES materials (id_material);
```

```
ALTER TABLE material_consumptions
ADD CONSTRAINT FK_material_consumptions_id_service FOREIGN KEY (id_service)
REFERENCES services (id_service);
```

```
-- Services
```

```
ALTER TABLE services ADD CONSTRAINT FK_services_id_service_type
FOREIGN KEY (id_service_type)
REFERENCES service_types (id_service_type);
```

```
-- SalonSchedules
```

```
ALTER TABLE salon_schedules ADD CONSTRAINT FK_salon_schedules_id_salon
FOREIGN KEY (id_salon)
REFERENCES salons (id_salon);
```

```
-- DiscountCards
```

```
ALTER TABLE discount_cards ADD CONSTRAINT FK_discount_cards_id_client
FOREIGN KEY (id_client)
REFERENCES clients (id_client);
```

```
ALTER TABLE discount_cards ADD CONSTRAINT
FK_discount_cards_id_discount_card_type FOREIGN KEY (id_discount_card_type)
REFERENCES discount_card_types (id_discount_card_type);
```

```
--DiscountCardsTypes
```

```
ALTER TABLE discount_card_types ADD CONSTRAINT  
FK_discount_card_types_id_service_type FOREIGN KEY (id_service_type)  
REFERENCES service_types (id_service_type);
```

-- Appointments

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_client  
FOREIGN KEY (id_client)  
REFERENCES clients (id_client);
```

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_master  
FOREIGN KEY (id_master_specialization)  
REFERENCES master_specializations (id_master_specialization);
```

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_room  
FOREIGN KEY (id_room)  
REFERENCES rooms (id_room);
```

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_appointment_status  
FOREIGN KEY (id_appointment_status)  
REFERENCES appointment_statuses (id_appointment_status);
```

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_payment_type  
FOREIGN KEY (id_payment_type)  
REFERENCES payment_types (id_payment_type);
```

```
ALTER TABLE appointments ADD CONSTRAINT FK_appointments_id_discount_card  
FOREIGN KEY (id_discount_card)  
REFERENCES discount_cards (id_discount_card);
```

-- Supplies

```
ALTER TABLE supplies  
ADD CONSTRAINT FK_supplies_id_material FOREIGN KEY (id_material) REFERENCES  
materials (id_material);
```

```
ALTER TABLE supplies  
ADD CONSTRAINT FK_supplies_id_supplier FOREIGN KEY (id_supplier) REFERENCES  
suppliers (id_supplier);
```